notee.

Anote agora. Entenda depois.

PLANO DE PRODUTO & ENGENHARIA

Um projeto para construir e compreender.

Requisitos, experiência, arquitetura, operação local e uma trilha de implementação orientada por IA.

A IA orienta. Você implementa.

Um passo por vez, com explicações, testes e decisões que você consegue defender. Sem cronograma fixo e sem entregar o aprendizado a um gerador de código.

React + TypeScript / ASP.NET Core / PostgreSQL
Worker + SQS / Terraform + LocalStack / OpenTelemetry

VERSÃO 1.0 · 30 SETEMBRO 2026
Planejamento e protótipo; não uma aplicação de produção implementada.

GUIA DE NAVEGAÇÃO

Conteúdo

Produto, arquitetura e consistência

01 Como usar este plano 4

02 Decisões que não devem se perder 5

03 O produto e seu ciclo principal 6

04 Requisitos verificáveis 7

05 Mapa de navegação e telas 8

06 Design system Notee 9

07 Referência visual do protótipo 10

08 Notas, blocos e organização 11

09 Ancoragem de palavras e frases 12

10 Automação: conta e nota 13

11 Análise e sugestões assistidas 14

12 Pesquisa web: fluxo ponta a ponta 15

13 Fontes, limites e qualidade da resposta 16

14 Providers substituíveis, sem ficção 17

15 Publicação sem rede social 18

16 Arquitetura: limites antes de camadas 19

17 Solution, dependências e frontend 20

18 Modelo relacional: conta e notas 21

19 Modelo relacional: pesquisa e entrega 22

20 API: identidade, notas e revisões 23

21 API: dúvidas e publicações 24

22 Concorrência, revisões e idempotência 25

23 Outbox, SQS e Worker durável 26

24 Laboratório de falhas e garantias 27

Os links levam às seções no PDF. Para continuar com outra IA, use também a versão Markdown e o arquivo de estado do projeto.

GUIA DE NAVEGAÇÃO

Conteúdo

Operação, trilha e mentoria

25 Identidade e baseline de segurança 28

26 IA, dados privados e conteúdo externo 29

27 Testes de backend e verificação manual 30

28 Ambiente local e perfis de execução 31

29 LocalStack: capacidade e limites reais 32

30 Terraform e deploy exclusivamente local 33

31 CI/CD enxuto e verificável 34

32 Observabilidade e diagnóstico 35

33 Escala, busca, custo e recuperação 36

34 Marcos 01 a 03 37

35 Marcos 04 a 06 38

36 Marcos 07 a 09 39

37 Marcos 10 a 12 40

38 Marcos 13 a 15 41

39 Marcos 16 a 18 42

40 ADRs: decisões que você deve defender 43

41 Contrato da IA mentora 44

42 Prompt mestre para iniciar uma sessão 45

43 Continuidade entre sessões e modelos 46

44 Como usar o protótipo entregue 47

45 Aceite final e demonstração técnica 48

46 Referências 01 a 10 49

47 Referências 11 a 20 50

48 Referências 21 a 30 51

Os links levam às seções no PDF. Para continuar com outra IA, use também a versão Markdown e o arquivo de estado do projeto.

GUIA DE LEITURA

Como usar este plano

O Notee é um projeto pessoal de aprendizado: um bloco de notas com organização, publicação e pesquisa de dúvidas. Este documento define o produto-alvo e as decisões de engenharia; não representa um backend já implementado ou uma infraestrutura já validada.

Três materiais, três funções

Material	Como usar
Planejamento PDF / Markdown	Fonte de verdade para requisitos, arquitetura, limites, marcos e critérios de aceite.
Protótipo HTML	Referência visual e de navegação. Dados fictícios, armazenamento local e pesquisas simuladas.
Instruções para a IA mentora	Contrato pedagógico: explicar, orientar, aguardar sua implementação e revisar. Não construir o sistema por você.

Origem e autoridade das decisões

[B0] identifica o PDF original NoteNest, fornecido pelo autor. Foram preservados .NET, PostgreSQL, concorrência, idempotência, Outbox, Worker, testes e observabilidade. O documento original usa 12 sprints e descreve um produto privado; a aprovação desta conversa substitui essas escolhas por marcos sem datas, publicação e frontend completo. [B0, pp. 3–8]

As regras de produto aqui estabelecidas consolidam suas respostas. Detalhes não especificados, como limites iniciais, estados e contratos, são propostas técnicas deste plano, ajustáveis por ADR. [Rxx] indica documentação pública consultada em 30/09/2026; não é evidência de execução local.

Ordem recomendada

Leia primeiro o escopo, a automação e o contrato da IA. Navegue no HTML. Depois implemente um marco por vez. Ter profundidade significa explicar, testar e operar suas escolhas, não acumular tecnologias.

ESCOPO APROVADO

Decisões que não devem se perder

Tema	Decisão consolidada
Identidade	Notee. Slogan proposto: “Anote agora. Entenda depois.” Identidade própria, com referência no AbacatePay.
Frontend	React + TypeScript + Vite + React Router + TanStack Query. Tailwind como utilitário; tokens e componentes Notee.
Editor e organização	Editor de blocos. Uma categoria principal e várias tags por nota. Sugestões de dúvidas, tags e categoria precisam de aceite.
Pesquisa	Marcação manual de palavra/frase; análise do texto; pesquisa web; resultado separado; fontes ou “Sem fonte”.
Automação	Padrão da conta e override por nota. Análise e execução de pesquisa são controles diferentes.
Temas e providers	Produto generalista, sem viés religioso ou especialização em um tema. Providers/modelos substituíveis.
Conta	E-mail e senha; recuperação de senha desde cedo. Sem login social nesta etapa.
Compartilhamento	Privado, não listado e público; perfil e biblioteca somente para leitura. Sem rede social.
Backend e dados	Monólito modular .NET + API e Worker, PostgreSQL, Redis seletivo, histórico real de revisões.
Infraestrutura	Terraform e serviços AWS somente no LocalStack. Nenhuma conta AWS real é requisito do projeto.
Qualidade	Testes de backend e CI/CD enxuto. Sem testes automatizados de frontend agora. Responsividade fica para depois.
Aprendizado	Marcos sem cronograma. A IA explica os motivos antes; perguntas de verificação apenas quando solicitadas.

Fora do escopo desta etapa

Curtidas, comentários, seguidores, feed personalizado, colaboração simultânea, aplicativo mobile, modo offline completo, pagamentos, Kubernetes e microsserviços. Embeddings entram apenas como evolução opcional.

PRODUTO E EXPERIÊNCIA

O produto e seu ciclo principal

Problema: perguntas surgem durante a escrita, mas pesquisar imediatamente interrompe o raciocínio; deixar para depois faz essas perguntas se perderem. O Notee registra a dúvida com seu contexto e organiza uma pesquisa rastreável, sem modificar silenciosamente o texto do autor.

Vocabulário do produto

Nota é um documento do usuário. Bloco é uma unidade interna do editor, como parágrafo, título ou código. Categoria organiza notas; tag conecta assuntos. Não haverá uma entidade adicional “caderno” nesta versão. Dúvida, pesquisa, resposta e publicação são registros diferentes.

Jornada	Comportamento esperado
Escrever e organizar	Criar conta, recuperar acesso, criar nota, editar blocos e atribuir categoria/tags. Nada é público por padrão.
Capturar a dúvida	Selecionar palavra/frase ou aceitar uma sugestão. Confirmar a pergunta interpretada e preservar o contexto da revisão.
Pesquisar sem bloquear	Iniciar manualmente ou seguir a automação. Receber um identificador do trabalho e continuar usando a aplicação.
Revisitar a resposta	Abrir resultado separado, fontes, limitações e data. Refinar ou pesquisar novamente sem apagar tentativas anteriores.
Compartilhar conscientemente	Publicar um snapshot revisado. Respostas da IA só acompanham o snapshot quando o autor autorizar explicitamente.

O que demonstra sucesso

Uma pessoa consegue registrar uma pergunta sem sair da escrita, reencontrá-la, distinguir informação sustentada de resposta sem fonte e compartilhar somente o que escolheu. Para o aprendizado, o autor explica o caminho HTTP → transação → fila → Worker → resposta e reproduz suas falhas.

Não prometer

A IA não é um verificador infalível. O aplicativo organiza investigação e evidências, não entrega garantia de verdade, aconselhamento especializado ou um índice automático de certeza.

PRODUTO E EXPERIÊNCIA

Requisitos verificáveis

ID	Requisito	Evidência de aceite
RF01	Conta e recuperação	Fluxos completos com resposta anti-enumeração; token expirado/reutilizado rejeitado.
RF02	Nota em blocos	Criar, editar, arquivar, mover para lixeira e restaurar, sempre por ownership.
RF03	Categoria e tags	Categoria opcional única por nota; tags múltiplas, normalizadas e do mesmo dono.
RF04	Dúvida manual	Palavra ou frase vira uma dúvida com trecho, bloco, revisão e contexto.
RF05	Sugestões	Detectar candidatas; aceitar/ignorar; sugestão obsoleta não altera a nota.
RF06	Automação	Matriz conta/nota respeitada; nenhum job para sugestão ainda não aceita.
RF07	Pesquisa durável	HTTP 202, status persistido, retomada e cancelamento cooperativo.
RF08	Resposta rastreável	Fontes por afirmação; data, contexto e limitações; “Sem fonte” sem evidência utilizável.
RF09	Provider substituível	Fake determinístico e um adaptador real opt-in, sem SDK do fornecedor no domínio.
RF10	Publicação	Snapshot; privado/não listado/público; retirada de acesso sem expor o rascunho.
RF11	Histórico	Revisões imutáveis, conflito de edição e restauração como nova revisão.
RF12	Busca interna	Título/texto/tags/categoria com isolamento, paginação e FTS.
RF13	Controles e diagnóstico	Limites de utilização e telemetria sem conteúdo privado.
RF14	Ambiente reprodutível	Terraform/LocalStack, pipelines leves e runbook de inicialização e restauração.

Requisitos não funcionais transversais: isolamento entre usuários, conteúdo privado por padrão, acessibilidade básica, processamento assíncrono recuperável, uso limitado de recursos e operação demonstrável. Metas quantitativas serão hipóteses de laboratório, não SLAs comerciais.

PRODUTO E EXPERIÊNCIA

Mapa de navegação e telas

Área	Telas / estados	Objetivo
Entrada	Landing, login, cadastro	Apresentar o produto e acessar uma conta local.
Recuperação	Solicitar link, redefinir senha	Mensagem genérica; sucesso, token inválido e expirado.
Espaço pessoal	Visão geral, minhas notas	Acesso rápido ao que escrever e ao que revisar.
Escrita	Editor, seleção, modal de dúvida	Blocos, categoria, tags, sugestões e preferência local.
Organização	Categorias, busca global	Criar e editar categorias, filtrar e reencontrar ideias.
Pesquisa	Central de dúvidas, detalhe	Fila, execução, resultado, Sem fonte, falha e cancelamento.
Versões	Histórico e conflito	Comparar, preservar rascunho e restaurar revisão.
Compartilhar	Publicar, minhas publicações	Escolher snapshot e visibilidade; revisar inclusão de IA.
Leitura pública	Nota pública, perfil, biblioteca	Somente leitura de snapshots autorizados. Não é rede social.
Ciclo de vida	Arquivo, lixeira	Restaurar ou excluir com confirmação proporcional ao risco.
Controle	Configurações, diagnóstico	Conta, automação, privacidade e visão operacional restrita.
Referência	Design system, mapa de telas	Tokens, componentes e inventário de navegação.

Estados também são parte da interface

Planejar vazio, carregando, salvando, salvo, falha, sem permissão, conflito, limite atingido, contexto insuficiente e resultado desatualizado. Nunca simular progresso percentual exato sem informação do backend; etapas conhecidas são suficientes. Um erro de pesquisa não deve impedir a edição da nota.

O protótipo inclui 24 destinos de navegação contando variantes de pesquisa, além dos modais. No React, separar os estados da tela da lógica de domínio. O menu “Telas” do HTML é um recurso de demonstração, não uma permissão de acesso.

PRODUTO E EXPERIÊNCIA

Design system Notee

Direção proposta: natural, silenciosa e editorial. Referência declarada: a simplicidade da página pública do AbacatePay; sem copiar marca, textos ou dashboard autenticado. O Notee tem monograma tipográfico próprio, verde floresta e lima suave como acento. [R01]

Token	Valor inicial	Uso
brand / strong	#21573D / #17412D	Ação principal, foco e identidade.
accent	#D8F4A1	Ênfase discreta; usar texto escuro.
canvas / surface	#F7F8F5 / #FFFFFF	Fundo de trabalho e cards.
ink / muted	#1E2D25 / #6C7A71	Hierarquia textual. Validar contraste na combinação final.
border	#E1E7DE	Divisores e agrupamento visual.
spacing	4 / 8 / 12 / 16 / 24 / 32 / 48 px	Escala comum; sem valores aleatórios por componente.
radius	8 / 10 / 16 px	Controles, chamadas e cards.
type	UI 14 px; leitura 16 px; títulos 24–36 px	Fonte de sistema inicialmente. Ajustar legibilidade, não apenas densidade.

Construir um sistema pequeno e real

Começar com Button, TextField, Select, Checkbox, Switch, Badge, Dialog, Toast, Tabs, Card, EmptyState, Skeleton e SourceCard. Definir estados, foco, tamanhos e mensagens antes de espalhar CSS. Adotar primitivas acessíveis onde necessário; não reinventar gerenciamento de foco só para demonstrar complexidade.

Os tokens são a fonte; Tailwind é uma ferramenta de aplicação. Manter uma página de componentes no próprio app, sem exigir Storybook neste momento. Validar teclado, nome acessível, contraste e leitura de mensagens manualmente. Responsividade e testes automatizados do frontend ficam em evolução posterior.

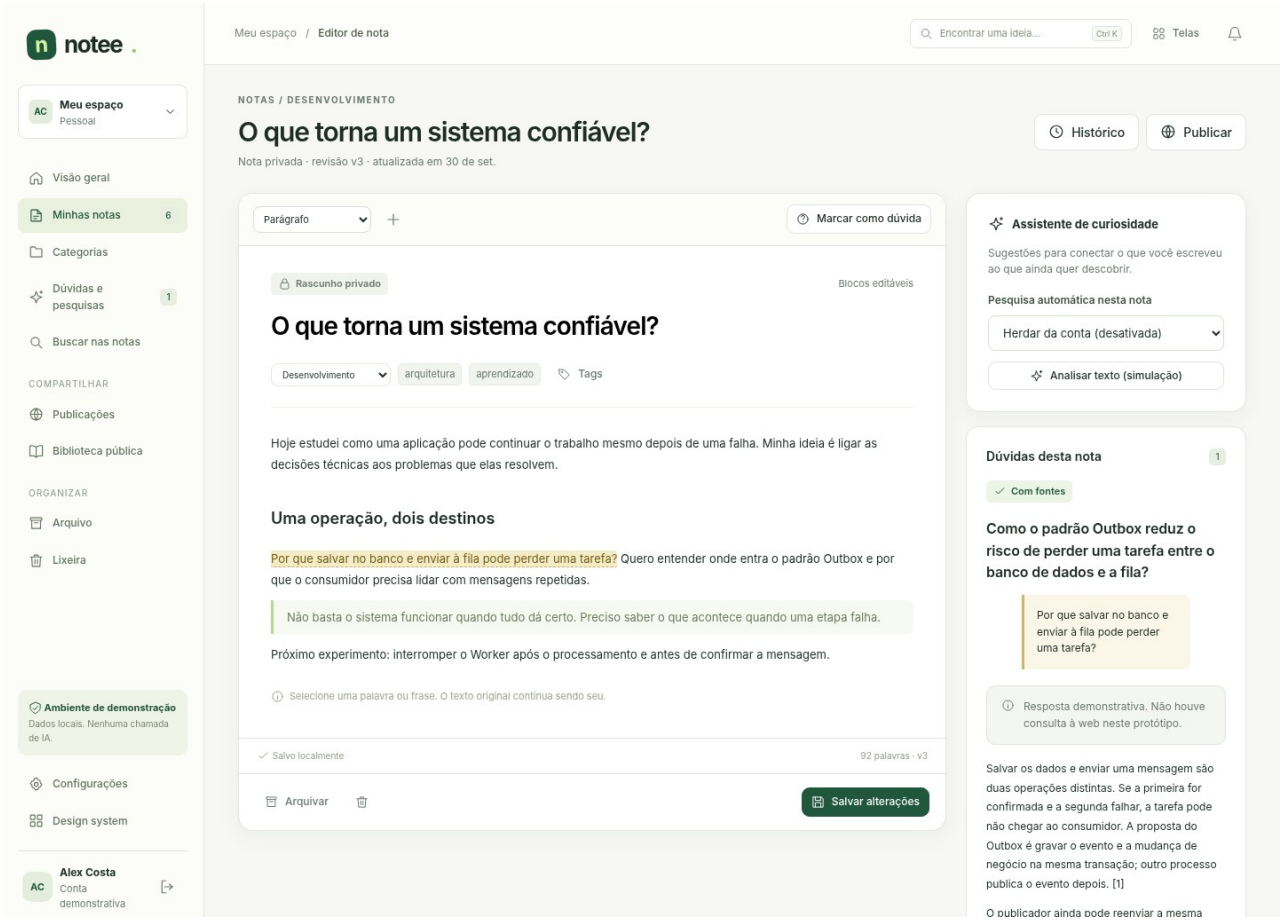
Regra de linguagem

Usar “Com fontes”, “Evidência parcial” e “Sem fonte”. Não usar um selo de certeza para uma síntese gerada por IA. Mensagens de erro explicam o próximo passo, sem culpar a pessoa.

PRODUTO E EXPERIÊNCIA

Referência visual do protótipo

A escrita ocupa o centro; a pesquisa fica em um painel separado. O destaque de dúvida pertence ao texto selecionado, enquanto a resposta pertence a outro registro. Esta imagem é uma captura do HTML entregue, com dados de demonstração.



Editor Notee: texto autoral, preferência por nota e painel de pesquisa. Captura do protótipo, não do produto implementado.

Pontos de atenção na migração

O painel lateral não deve roubar o foco do editor a cada atualização de status. Preservar seleção ao abrir o modal e oferecer atalho de teclado. O status de salvamento pertence à nota; o status de pesquisa pertence ao job. Não misturar os dois estados.

O HTML usa DOM e dados locais para demonstrar a experiência. No React, o buffer de edição fica no editor, os dados remotos ficam no TanStack Query e a API decide permissões. Nunca portar o localStorage de demonstração como armazenamento de autenticação ou fonte de verdade do produto.

PRODUTO E EXPERIÊNCIA

Notas, blocos e organização

Modelo inicial de edição

Proposta: Tiptap/ProseMirror com extensões básicas e IDs estáveis de bloco; o backend aceita um schema JSON versionado e validado. Começar com parágrafo, heading, lista, citação e código. Não depender de serviços pagos de IA, colaboração ou histórico do editor: esses comportamentos pertencem ao Notee. [R06, R07]

Conceito	Regra proposta
Note	Id, OwnerId, Title, ContentSchemaVersion, ContentJson, PlainText, Version, Status, CategoryId e timestamps UTC.
Block	Id estável, tipo permitido, conteúdo e atributos limitados. Mover o bloco não troca sua identidade.
Category	Pertence a um usuário. Nome normalizado único por dono. Categoria opcional; sem árvore nesta versão.
Tag	Também pertence ao usuário. Relação N:N com Note; rejeitar associação com taxonomia de outro dono.
NoteRevision	Snapshot imutável de conteúdo e metadados; restaurar cria uma revisão mais nova.
NoteStatus	Active, Archived, Trashed. Só Active permite editar, analisar, pesquisar ou publicar.

Limites iniciais sugeridos, não requisitos definitivos: título até 160 caracteres, 200 blocos, 100 mil caracteres de texto e 12 tags por nota. Validar a mesma política na API; o limite de contexto enviado ao modelo deve ser muito menor que o tamanho permitido da nota.

Remoção e restauração

Excluir categoria deixa notas sem categoria, sem apagar conteúdo. Excluir tag remove associações. Mover nota para arquivo/lixeira cancela jobs novos e retira publicações nesta versão. Restaurar a nota não republica nem reinicia pesquisas automaticamente. Exclusão definitiva exige que a nota já esteja na lixeira.

PRODUTO E EXPERIÊNCIA

Ancoragem de palavras e frases

Uma dúvida pode nascer de uma palavra isolada ou de uma frase. Guardar apenas offsets absolutos do documento é frágil: inserir texto antes desloca a seleção. O objeto Anchor preserva identidade, posição e evidência do trecho; não depende exclusivamente da aparência do destaque.

```
DoubtAnchor
  noteId, noteRevisionId, blockId
  fromUtf16, toUtf16          // fim exclusivo
  exactQuote, prefix, suffix
  contextSnapshotHash, anchorVersion
  status: Attached | Stale | Orphaned
```

Política proposta

No primeiro recorte, a seleção fica dentro de um único bloco textual; o sistema explica esse limite para seleções que cruzam blocos. O intervalo usa unidades UTF-16 sobre o texto simples do bloco, igual na API e no cliente, sem cortar pares substitutos ou grafemas. Não confundir offsets locais de texto com as posições estruturais do ProseMirror.

Enquanto o editor está aberto, mapear a marca pelas transações de edição. Após recarregar, reancorar por BlockId + citação + contexto. Se houver uma única correspondência verificável, registrar nova versão da âncora. Havendo ambiguidade ou remoção, mostrar “Trecho alterado” e pedir remarcação; não adivinhar o destino.

Caso	Comportamento de aceite
Palavra repetida	Usar contexto anterior/posterior; nunca selecionar automaticamente a primeira ocorrência.
Bloco movido	Preservar o vínculo pelo BlockId; validar se o texto relevante permaneceu igual.
Nota editada durante pesquisa	Concluir contra o snapshot de origem. Marcar o resultado como referente à revisão antiga.
Bloco apagado	Dúvida continua no histórico com âncora órfã. Não mover resposta para outro texto.

Teste que revela maturidade

Usar acentos, emojis, duas ocorrências da mesma palavra, edição antes do trecho e remoção do bloco. Validar os offsets no backend contra a revisão persistida, não confiar no trecho enviado pelo navegador.

PRODUTO E EXPERIÊNCIA

Automação: conta e nota

Separar três decisões: analisar texto para sugerir, aceitar uma sugestão como dúvida e executar uma pesquisa. Ativar a pesquisa automática não autoriza transformar toda frase em uma pergunta aceita, nem compartilhar todo o conteúdo com um fornecedor.

AutoResearch da conta	Override da nota	Após dúvida aceita e contexto suficiente
Desativado	Herdar	Aguardar a ação Pesquisar.
Ativado	Herdar	Enfileirar automaticamente, se as demais proteções permitirem.
Qualquer	Ativar	Enfileirar automaticamente, respeitando consentimento e limites.
Qualquer	Desativar	Somente pesquisa manual; a marcação continua disponível.

Precedência e defaults

Defaults reais: notas privadas, pesquisa automática desligada e nenhum envio externo sem consentimento específico. A análise externa tem seu próprio controle. O padrão da nota é Herdar. Uma chave global de bloqueio externo tem precedência sobre todos os overrides; uma nota marcada como “não enviar a serviços externos” também veta análise e pesquisa externas.

```
MayEnqueue = OwnerAuthorized AND Note.Active
  AND Doubt.Accepted AND ContextSufficient
  AND ExternalConsent AND NOT GlobalBlock
  AND NOT Note.ExternalBlock AND WithinBudget
  AND NoEquivalentActiveJob

Auto = MayEnqueue AND EffectiveAutoResearch
```

Pesquisa manual independe de EffectiveAutoResearch, mas obedece aos mesmos limites, permissões e consentimentos. Revalidar esses controles antes de cada chamada externa: um job na fila não congela para sempre a autorização do usuário. Desativar solicita cancelamento cooperativo; não desfaz uma requisição já enviada.

Evitar surpresas e custos

Mudar a preferência não dispara o acervo antigo. Oferecer, no futuro, “Pesquisar pendentes” com quantidade, estimativa e confirmação. Deduplicar por dúvida, revisão e configuração relevante; uma nova tentativa precisa de intenção explícita.

PRODUTO E EXPERIÊNCIA

Análise e sugestões assistidas

Primeiro reconhecer, depois sugerir

A análise observa uma revisão salva, não cada tecla. Começar com o botão Analisar; evoluir para execução após pausa de digitação e salvamento quando a pessoa habilitar. Proposta inicial: debounce de 3 segundos, tamanho mínimo de mudança e no máximo uma análise ativa por nota. Esses números devem ser medidos e ajustados.

Saída	Conteúdo mínimo	Aplicação
DoubtSuggestion	Bloco, trecho exato, pergunta sugerida, motivo breve e revisão.	Aceitar cria Doubt; ignorar registra dismiss; nenhuma pesquisa antes do aceite.
CategorySuggestion	CategoryId existente do dono ou proposta de nome, com justificativa.	Confirmar troca de categoria. Criar categoria nova exige intenção do usuário.
TagSuggestion	Tags normalizadas; distinguir existentes e novas.	Aceitar individualmente ou em grupo selecionado, sem duplicar associações.

Persistir AnalysisRun com NoteRevisionId, modelo/provider, versão de prompt, custo reportado e resultado estruturado. Um retorno de revisão antiga deve aparecer como obsoleto ou ser descartado; não pode sobrescrever escolhas recentes. Uma sugestão ignorada para o mesmo trecho/contexto não deve reaparecer a cada salvamento.

Palavra sem pergunta explícita

O contexto próximo ajuda a formular a pergunta, mas não elimina ambiguidade. Selecionar “Java” pode significar linguagem, ilha ou café. Mostrar a interpretação antes da pesquisa; quando houver mais de uma leitura plausível, usar NeedsContext e aguardar um complemento mesmo com automação ativa.

Controle sem uma segunda rede social

O motor não deve inferir crenças, posicionamento político ou perfis sensíveis para classificar o usuário. Classifica conteúdo sob seu controle e com taxonomia da sua conta. Nenhum tema tem status de feature especial.

PRODUTO E EXPERIÊNCIA

Pesquisa web: fluxo ponta a ponta

Etapa	Responsabilidade	Registro durável
1. Preparar	Validar dono, pergunta, revisão, consentimento, limite e deduplicação.	ResearchJob + snapshot do contexto + Outbox na mesma transação.
2. Enfileirar	Publicador entrega comando ao SQS; API já respondeu 202.	EventId estável; JobId e trace context; sem texto privado no payload.
3. Interpretar	Confirmar pergunta e escopo; interromper se faltar contexto.	QueryPlan limitado e versão da política de pesquisa.
4. Buscar	Executar consultas web e obter fontes permitidas.	Consultas minimizadas, IDs do provider, URLs e metadados.
5. Sintetizar	Gerar resposta somente a partir das evidências disponíveis.	Resposta estruturada, afirmações e IDs de fontes.
6. Validar e salvar	Validar schema, citações, estado da nota e cancelamento.	ResearchAnswer + evidências + status + inbox em commit.
7. Apresentar	Cliente consulta status; exibe resposta para revisão do autor.	Notificação interna e status de revisão independentes do job.

Implementar limites de consultas, páginas, tamanho de resposta, tempo e chamadas por job. Começar com polling de status em intervalos de poucos segundos apenas enquanto houver atividade; parar em estado terminal e reduzir frequência com a aba oculta. SSE ou SignalR não são necessários neste recorte.

Estados que não devem ser confundidos

```
Job: Queued -> Running -> Succeeded
                    |-> RetryScheduled -> Queued
                    |-> Failed
                    |-> CancelRequested -> Cancelled

Evidence: Sourced | Partial | NoSource
Review: Pending | Reviewed | Rejected
```

Succeeded com NoSource é uma execução concluída sem evidência utilizável; não é um erro de infraestrutura. Cancelamento é cooperativo. O navegador fechar não cancela automaticamente o trabalho real do Worker.

PRODUTO E EXPERIÊNCIA

Fontes, limites e qualidade da resposta

Cada afirmação factual relevante precisa apontar para a evidência que a sustenta. Uma lista de links no fim não basta para demonstrar suporte. Fonte encontrada e fonte utilizada são coleções diferentes; ambas podem ser registradas, com retenção limitada.

Registro	Campos propostos
Source	Id, URL canônica, título, editor, data de publicação quando disponível, acesso UTC e estado de acesso.
Evidence	SourceId, trecho curto permitido ou referência recuperável, hash, localizador e método de obtenção.
ClaimCitation	ClaimId, SourceId e relação: suporte, contexto ou divergência.
Answer	Resumo, desenvolvimento, limites, lacunas, data da pesquisa, snapshot da pergunta e versão do modelo/prompt.

Política de evidência

Priorizar fontes primárias adequadas ao assunto e diversificar quando houver divergências relevantes. Separar texto documentado, interpretação e hipótese. Para temas atuais, registrar a data do fato e da pesquisa. Não deduzir que um conteúdo é recente somente pela data do acesso.

Sem suporte utilizável, exibir literalmente “Sem fonte” e explicar o que faltou. Uma fonte inacessível ou um snippet sem contexto não deve ser apresentado como leitura integral. Havendo suporte somente parcial, mostrar “Evidência parcial” e marcar os trechos sem suporte. Ausência de evidência não prova falsidade.

Não aceitar URLs inventadas pelo modelo: citações devem referenciar IDs previamente obtidos da busca. Validar que o link existe no conjunto recuperado não demonstra, sozinho, que a fonte sustenta a conclusão. Usar avaliação humana e casos de teste de suporte para essa segunda camada. APIs com busca nativa podem retornar anotações e fontes; o adaptador deve preservá-las. [R08]

Mesmo produto para todos os assuntos

Não criar um modo religioso nem um modo de persuasão política. O aplicativo apresenta contexto, fontes e divergências com neutralidade, sem orientar crenças ou escolhas eleitorais. Evitar notas percentuais de “confiança” não calibradas.

ARQUITETURA DE PESQUISA

Providers substituíveis, sem ficção

Provider-agnostic significa preservar contratos do Notee e isolar integrações; não significa que todos os modelos tenham as mesmas capacidades ou produzam o mesmo resultado. Começar com FakeResearchProvider determinístico. Escolher um único fornecedor real quando houver acesso, consentimento e um teto de custo. Nenhum nome de modelo é fixado neste plano.

Porta de aplicação	Contrato esperado
ITextAnalysisProvider	Sugestões estruturadas de dúvida e taxonomia, com trecho exato e revisão.
IWebSearchProvider	Consultas limitadas e resultados com IDs, URLs, metadados e evidências recuperadas.
IAnswerSynthesisProvider	QuestionContext + EvidenceSet → AnswerDraft com ClaimIds/SourceIds; sem URLs livres.
IResearchProvider	Fachada opcional para fornecedores com busca e síntese nativas; devolver o mesmo contrato final.
ProviderCapabilities	Busca real, citações, saída estruturada, cancelamento, idempotência, uso reportado e job remoto.

Infrastructure traduz DTOs, SDKs, anotações e erros. Application coordena limites, consentimento, estado e retomada. Domain desconhece o fornecedor. Registrar ProviderId, ModelId, PromptVersion, PolicyVersion e SearchMode em cada execução para permitir auditoria.

Erros e capacidades explícitos

Normalizar AuthFailure, QuotaExceeded, RateLimited, Timeout, TransientFailure, InvalidOutput, UnsupportedCapability e UnknownOutcome. Retry deve obedecer ao tipo de operação e ao efeito externo; não repetir toda falha cegamente. Se o modelo não tiver busca, combiná-lo com um provedor de busca ou rejeitar o modo, em vez de chamar conhecimento paramétrico de “pesquisa na internet”.

Privacidade e dependências externas

LocalStack não emula ChatGPT, Gemini, Claude nem a internet. O perfil Fake funciona sem APIs comerciais. O perfil Live depende de fornecedor externo e pode gerar custos. Não trocar de fornecedor silenciosamente em um fallback; isso pode mudar o destinatário dos dados.

PRODUTO E EXPERIÊNCIA

Publicação sem rede social

A nota é o rascunho privado; Publication é um snapshot autorizado. O endpoint público nunca serializa diretamente a entidade Note. Construir um DTO público com allowlist de campos, conteúdo validado e respostas explicitamente incluídas.

Visibilidade	Acesso e descoberta
Private	Sem publicação acessível anonimamente. Leitura autenticada pelo dono.
Unlisted	Link com token aleatório opaco. Não aparece na biblioteca/perfil; noindex é uma instrução de indexação, não segurança.
Public	URL por handle/slug; aparece em biblioteca e perfil público de leitura. Sem ranking, seguidores ou comentários.

Publicar é um comando, não um efeito de autosave

O autor escolhe revisão, título, visibilidade e respostas revisadas que deseja incluir. Copiar esses dados para o snapshot na transação de publicação. A resposta da IA não entra por padrão; ao incluir conteúdo sem evidência, preservar o rótulo “Sem fonte” e pedir confirmação informada.

Editar ou restaurar o rascunho não atualiza o publicado. Um novo Publish/UpdatePublication cria nova revisão publicada. Unpublish revoga o acesso e invalida caches; arquivo/lixeira revogam a publicação nesta versão. Um link já compartilhado pode ter sido copiado por terceiros; retirada do ar não desfaz cópias externas.

Segurança e concorrência entre módulos

Validar owner e revisão no comando. Na leitura pública, verificar se a publicação permanece ativa antes de devolver o snapshot; não confiar em um cache antigo para autorizar. Impedir a corrida publicar-versus-excluir com uma política transacional documentada. Metadados privados, e-mail, prompts e outras perguntas não incluídas jamais integram o DTO.

SEO como decisão, não requisito oculto

A SPA React atende à primeira versão. Pré-visualização de links e indexação completa podem requerer HTML público renderizado no servidor ou pré-gerado. Registrar isso como evolução, sem adicionar Next.js ao escopo aprovado.

ENGENHARIA DO SISTEMA

Arquitetura: limites antes de camadas

Manter um monólito modular, com dois hosts: API e Worker. Módulos de negócio compartilham o processo HTTP e um PostgreSQL inicialmente, mas possuem contratos e escrita de dados bem definidos. Ter pastas Domain/Application/Infrastructure, por si só, não estabelece esses limites. A base do NoteNest é preservada e ganha organização por capacidade de negócio. [B0, pp. 3–4]

Módulo	Responsabilidade	Não deve assumir
Identity	Contas, credenciais, sessões, recuperação e preferências globais.	Conteúdo de notas ou resultados da pesquisa.
Notes	Documentos, blocos, categorias, tags, revisões e ciclo de vida.	SDK de IA, fila ou integração de pesquisa.
Research	Dúvidas, sugestões, jobs, evidências, respostas e orçamento.	Reescrever texto autoral ou publicar automaticamente.
Publishing	Snapshots, slugs, visibilidade, perfil e catálogo de leitura.	Expor a tabela privada de notas como resposta pública.
Infraestrutura compartilhada	Relógio, transações, Outbox, telemetria e primitivas pequenas.	Um módulo genérico com todas as regras do sistema.

```
Browser / React
  -> Notee.Api -> contratos de aplicacao dos modulos
                  -> PostgreSQL (fonte de verdade)
                  -> Redis (cache seletivo)

PostgreSQL.Outbox -> SQS / LocalStack -> Notee.Worker
  -> Research -> Search / LLM opt-in -> PostgreSQL

API + Worker -> OpenTelemetry -> observabilidade local
```

Domain contém regras sem ASP.NET, EF Core, Redis ou AWS. Application coordena casos de uso e portas. Infrastructure implementa integrações. Endpoints traduzem HTTP e não decidem regras de negócio. Uma consulta entre módulos passa por contrato explícito; evitar escrita direta nas tabelas do vizinho.

O primeiro fluxo não precisa de tudo

Comece com Notes e testes. API, persistência e identidade entram incrementalmente. Research é a justificativa real para a fila e o Worker; o webhook genérico do material original vira exercício opcional posterior.

ENGENHARIA DO SISTEMA

Solution, dependências e frontend

```
notee/
  src/
    Notee.Api/
    Notee.Worker/
    Notee.BuildingBlocks/
    Modules/
      Identity/{Domain,Application,Infrastructure,Contracts}/
      Notes/{Domain,Application,Infrastructure,Contracts}/
      Research/{Domain,Application,Infrastructure,Contracts}/
      Publishing/{Domain,Application,Infrastructure,Contracts}/
  web/src/{app,features,shared}/
  tests/{Domain,Application,Integration,Architecture}/
  infra/terraform/{core,full-localstack}/
  docs/{adr,runbooks,learning,evaluations}/
  compose.yaml
  .github/workflows/ci.yml
```

Começar com poucos projects .NET e namespaces internos por módulo; extrair assemblies quando a fronteira estiver clara. A árvore acima descreve responsabilidades, não obriga dezenas de bibliotecas vazias no primeiro commit. Restringir tipos internos e criar testes arquiteturais para dependências proibidas.

Camada	Pode depender de	Evitar
Domain	Primitivas mínimas e tipos do próprio módulo.	Frameworks web, SDKs e implementações de persistência.
Application	Domain e Contracts explícitos.	Controllers ou DbContext concreto.
Infrastructure	Application/Domain e bibliotecas externas.	API e UI.
API / Worker	Composition roots e casos de uso.	Regras duplicadas entre hosts.

No frontend: app contém rotas e providers; features separa notes, research, auth e publishing; shared/ui concentra o design system. TanStack Query cuida do estado remoto, invalidação e polling. O conteúdo ainda não salvo é estado local do editor, não uma segunda cópia indiscriminada do cache. [R05]

Fixar .NET 10/C# 14 como baseline e usar patches suportados; registrar SDK em global.json, dependências em arquivos de lock e versões de ferramentas no README. Para React/Vite/Router, escolher uma combinação estável compatível e verificar a exigência de Node na instalação; não usar latest no CI sem lock. [R02–R04]

DADOS E CONTRATOS

Modelo relacional: conta e notas

Schemas por módulo ajudam a visualizar propriedade. Começar com uma instância PostgreSQL e migrations versionadas do EF Core. Schema JSON define o documento do editor; tabelas relacionais mantêm identidade, relações e invariantes pesquisáveis.

Tabela / responsabilidade	Campos e restrições relevantes
identity.users	Identity user, e-mail normalizado único, nome público opcional e timestamps. Não armazenar senha em claro.
identity.sessions	SessionId, UserId, família de refresh, hash do token, expiração, revogação e último uso.
identity.preferences	UserId único, AutoResearchDefault, AnalysisMode, bloqueio externo, consentimento e versão da política aceita.
notes.notes	Id, OwnerId, Title, CategoryId opcional, ContentJson JSONB, PlainText, SchemaVersion, Version BIGINT, Status e timestamps.
notes.revisions	Id, NoteId, OwnerId, NoteVersion, snapshot e motivo. UNIQUE(NoteId, NoteVersion). Imutável.
notes.categories	Id, OwnerId, Name e NormalizedName. UNIQUE(OwnerId, NormalizedName).
notes.tags / note_tags	Taxonomia por dono; UNIQUE(NoteId, TagId). Impedir vínculos com tags de outro dono.
notes.settings	NoteId único, AutoResearchOverride: Inherit/On/Off, ExternalBlock e versão da preferência.

Índices guiados pelas consultas

Listagem privada: (OwnerId, Status, UpdatedAt DESC, Id). Revisões: (NoteId, NoteVersion DESC). Categorias/tags: unicidade normalizada por OwnerId. Normalização de espaços e caixa deve ser definida; não presumir que todas as collation rules sejam idênticas entre ambientes.

Version de concorrência e NoteRevision não são a mesma coisa: uma protege a escrita, a outra preserva história. Gerar PlainText a partir do JSON validado no servidor para busca e contexto; não aceitar texto extraído do navegador como representação autoritativa.

Defesa em camadas

Filtrar todas as operações privadas por dono. Onde for viável, usar FKs compostas com OwnerId para proteger associações. Testes devem provar que IDs de outra conta não permitem ler, associar, restaurar ou publicar dados.

DADOS E CONTRATOS

Modelo relacional: pesquisa e entrega

Tabela	Propósito e invariante
research.doubts	OwnerId, NoteId, Anchor, Question, origem manual/aceita, status e versão. Aceite é explícito.
research.analysis_runs / suggestions	Revisão analisada, propostas e estado Accepted/Dismissed/Stale. Deduplicar por trecho/contexto.
research.jobs	JobId, DoubtId, ContextRevisionId, estado, prioridade, configuração congelada, lease, fencing token, cancelamento e timestamps.
research.attempts / steps	Tentativas e checkpoints com status, request remoto, erro normalizado e uso reportado. Não armazenar raciocínio interno do modelo.
research.answers / sources / citations	Resposta imutável por execução, evidência, links e revisão humana. Fonte deve pertencer ao conjunto dessa pesquisa.
research.budget_reservations	Reserva atômica por usuário/período/job; consumo, liberação e eventual resultado financeiro desconhecido.
publishing.publications / revisions	NoteId, autor público, visibilidade, slug/token, estado e snapshot selecionado. UNIQUE(OwnerId, Slug) para slugs públicos.
infra.outbox_messages	EventId, tipo, SchemaVersion, payload mínimo, ocorrência, lease e publicação.
infra.inbox_messages	UNIQUE(ConsumerName, EventId). Confirma o efeito local concluído, não só o recebimento.
infra.idempotency_records	OwnerId, Operation, KeyHash, RequestHash, status e resposta. UNIQUE no escopo da operação.

Índices candidatos: jobs por (Status, NextAttemptAt), lease expirado, e (OwnerId, CreatedAt, Id); Outbox por pendência/CreatedAt; respostas por DoubtId/CreatedAt; publicações por Visibility/PublishedAt/Id. Medir planos reais antes de adicionar índices para todo filtro.

Usar migrações de schema e dados reversíveis quando possível, scripts revisados e seed apenas fictício. A partir de API + Worker simultâneos, migrations passam a ser uma etapa controlada de deploy, não uma corrida no startup de cada host.

DADOS E CONTRATOS

API: identidade, notas e revisões

Contrato proposto REST /api/v1 com OpenAPI gerado do código. O cliente TypeScript pode ser gerado do contrato quando o fluxo estabilizar; não manter DTOs divergentes manualmente como prática permanente. Todos os endpoints privados derivam OwnerId da identidade autenticada.

Grupo	Rotas propostas
Autenticação	POST /auth/register, /auth/login, /auth/refresh, /auth/logout. GET /me.
Recuperação	POST /auth/forgot-password; POST /auth/reset-password. Resposta genérica na solicitação.
Notas	GET /notes?query=&categoryId=&tag=&status=&cursor=&limit= ; POST /notes.
Nota individual	GET /notes/{id}; PUT /notes/{id} com expectedVersion.
Ciclo de vida	POST /notes/{id}/archive, /trash, /restore; DELETE /notes/{id} apenas quando Trashed.
Histórico	GET /notes/{id}/revisions; GET /notes/{id}/revisions/{revisionId}; POST /notes/{id}/restore-revision.
Taxonomia	GET/POST /categories e /tags; PATCH/DELETE /categories/{id} e /tags/{id}.
Preferências	GET/PATCH /me/preferences; GET/PATCH /notes/{id}/settings.
Operação	GET /health/live, /health/ready e /openapi/v1.json. Diagnóstico protegido por política de operador.

Respostas consistentes

201 + Location para recursos criados; 200/204 para comandos concluídos; 202 + Location para trabalho assíncrono; 400/422 para validação conforme política documentada; 401 sem autenticação; 403 sem permissão; 404 para recurso ausente ou privado de outra conta; 409 para conflito de versão ou reutilização incompatível de chave; 429 para limite de requisições.

Erros usam Problem Details, com code estável e traceId seguro. Nunca devolver stack trace, chave, prompt ou conteúdo de outro usuário. Validar paginação, tamanho de corpo e limites no servidor. Cursor inclui a ordenação estável, mas não substitui o filtro de dono.

DADOS E CONTRATOS

API: dúvidas e publicações

Operação	Contrato proposto
Marcar dúvida	POST /notes/{id}/doubts: revisionId, blockId, intervalo, exactQuote e question. Idempotency-Key.
Analisar texto	POST /notes/{id}/analysis-runs → 202; GET /analysis-runs/{id}.
Aceitar sugestão	POST /suggestions/{id}/accept ou /dismiss; revisão obsoleta exige reanálise, não aceite cego.
Iniciar pesquisa	POST /doubts/{id}/research-jobs → 202 + Location. GET /research-jobs/{id}.
Cancelar / tentar de novo	POST /research-jobs/{id}/cancel; POST /doubts/{id}/research-jobs com nova chave e motivo de reexecução.
Revisar resposta	GET /doubts/{id}/answers; POST /answers/{id}/review. Revisão não reescreve o resultado.
Publicar	POST /notes/{id}/publications: revisionId, visibility, slug e includedAnswerIds.
Atualizar / retirar	POST /publications/{id}/revisions; DELETE /publications/{id} revoga acesso público.
Leitura pública	GET /public/notes/{handle}/{slug}; GET /shared/{token}; GET /public/profiles/{handle}; GET /public/notes.

```
POST /api/v1/doubts/{id}/research-jobs
Idempotency-Key: <chave-da-intencao-do-cliente>

202 Accepted
Location: /api/v1/research-jobs/<jobId>
{ "jobId": "...", "status": "queued" }
```

A semântica é importante: a API confirma que o trabalho foi aceito de forma durável, não que a resposta já existe. O cliente consulta o recurso de status e depois a resposta. Campos operacionais do provider não devem vazar no contrato de tela.

Diferenciar validação e indisponibilidade

Pergunta ambígua fica em NeedsContext antes do job; capacidade não suportada é erro claro de configuração. Quota de fornecedor e indisponibilidade não viram “Sem fonte”. A interface deve oferecer refinar, tentar novamente ou aguardar conforme a causa.

CONFIABILIDADE

Concorrência, revisões e idempotência

A base já prevê expectedVersion e conflito HTTP 409. Preservar essa semântica e implementá-la na condição de escrita do banco, não apenas em um if antes de SaveChanges. O EF Core oferece tokens de concorrência e sinaliza o conflito quando a linha esperada não pode ser atualizada. [B0, p. 5; R09]

```
Cliente le v7.
A envia expectedVersion=7 -> grava v8 + snapshot v8.
B envia expectedVersion=7 -> nenhuma linha corresponde -> 409.
A interface preserva o rascunho de B e oferece comparacao.
```

No PostgreSQL, usar Version BIGINT controlada pela aplicação e configurada como concurrency token; não transportar automaticamente o rowversion específico de SQL Server. Nota e nova NoteRevision são gravadas na mesma transação. Salvar conteúdo idêntico pode ser no-op; evitar versões por tecla. [R09]

Autosave seguro, em um recorte posterior do editor

Começar com Salvar explícito. Depois serializar as requisições de autosave por nota, consolidar mudanças durante uma gravação em andamento e usar a versão devolvida pelo servidor na próxima requisição. Nunca deixar uma resposta antiga do servidor substituir texto que o usuário digitou depois.

Situação	Regra para Idempotency-Key
Mesma chave + mesmo payload	Devolver a mesma criação/aceite dentro da janela de retenção.
Mesma chave + payload diferente	409 com código de chave incompatível.
Duas requisições simultâneas	Constraint única + transação arbitram; memória do processo não basta.
Recurso já apagado ou acesso revogado	Revalidar autorização; não reproduzir conteúdo privado antigo por cache de idempotência.

Não confundir garantias

Idempotência HTTP evita duplicar a mesma intenção no período configurado. Deduplicação de mensagens evita repetir efeitos locais. Nenhuma delas garante, sozinha, cobrança externa exatamente uma vez.

CONFIABILIDADE

Outbox, SQS e Worker durável

A Outbox resolve a falha de duas escritas independentes: Job + evento são confirmados no PostgreSQL; publicar no SQS ocorre depois. O SQS Standard pode entregar mensagens repetidas, portanto o consumo deve tolerar duplicatas. Essas são propriedades documentadas; o desenho abaixo é a aplicação proposta ao Notee. [R10, R11]

Componente	Responsabilidade e fronteira da transação
OutboxPublisher	Reivindicar lote com lease, publicar EventId estável e marcar publicado. Crash após enviar pode reenviar: comportamento esperado.
ResearchConsumer	Consultar inbox; adquirir lease do job por compare-and-swap. Não manter transação SQL aberta durante chamadas externas.
Checkpoints	Persistir etapas concluídas. Antes de repetir uma integração, verificar se existe resultado persistido ou request remoto consultável.
Finalização	Resposta + status terminal + Inbox(Consumer, EventId) em commit. Somente depois executar DeleteMessage no SQS.
Lease / fencing	Renovar a concessão e o visibility timeout enquanto houver trabalho. Toda gravação tardia valida o fencing token; um Worker antigo não sobrepõe o novo.

Uma autoridade de retry

Proposta para o marco avançado: em falha transitória conhecida, gravar RetryScheduled + NextAttemptAt e concluir o consumo daquele evento. Um dispatcher persistente cria, quando devido, um novo evento de tentativa via Outbox. Não combinar isso com republicações independentes em outros componentes. Redelivery do SQS continua protegendo crashes antes do commit.

Usar backoff com jitter e limites de tentativas/tempo; retries de HttpClient devem ser pequenos e restritos a operações seguras. DLQ trata mensagens venenosas ou falhas repetidas de consumo; falha funcional do job permanece visível no banco. Recebimentos SQS e tentativas de negócio são contadores distintos.

Por que isso não ocupa a API?

A API apenas valida e confirma o agendamento. O Worker é outro host/processo e executa I/O assíncrono com concorrência limitada. Job, checkpoint, lease e resultado ficam no PostgreSQL; SQS é transporte, não o banco interno de progresso. Não usar Task.Run no request para trabalho durável.

CONFIABILIDADE

Laboratório de falhas e garantias

Falha provocada	Resultado exigido	Evidência a coletar
Fila indisponível após commit	Job aceito e Outbox pendente; retomar publicação sem perder o trabalho.	Linha de Outbox, idade, log correlacionado e posterior consumo.
Crash após publicar, antes de marcar	Evento pode repetir; um único efeito local persistido.	EventId repetido e inbox deduplicada.
Crash do Worker antes de responder	Lease expira; outro Worker assume e retoma o último checkpoint confirmado.	Mudanca de fencing token; tentativa e etapa registradas.
Resposta salva, ACK SQS perdido	Redelivery encontra inbox terminal e confirma a mensagem sem recriar resposta.	Uma resposta persistida e redelivery benigno.
Provider respondeu; commit falhou	Resultado externo pode ser desconhecido. Recuperar por request remoto se possível; sinalizar risco de repetição/custo.	UnknownOutcome, ID externo e decisão de retry auditada.
Nota editada durante pesquisa	Resposta ligada ao snapshot original, marcada como desatualizada.	NoteRevisionId diferente do atual, sem overwrite.
Consentimento revogado / nota apagada	Bloquear próxima chamada; impedir persistência indevida do resultado; limpeza idempotente.	Cancelamento e teste de corrida com resposta tardia.
Provider 429, timeout ou payload inválido	Erros normalizados; retry limitado ou falha definitiva. Não rotular como Sem fonte.	Tentativas, espera, métrica e status final.

Limites que devem aparecer no README

A garantia pretendida é não perder trabalho confirmado, sob retenção e operação adequadas, e evitar efeitos locais duplicados. Não há promessa end-to-end de exactly-once em serviços externos. Se um provider não oferece idempotência ou consulta de resultado, uma falha entre resposta e commit pode exigir repetir a chamada.

Reprocessar DLQ exige primeiro corrigir a causa e verificar estado/autorização do job. Mover todas as mensagens de volta à fila sem critério não é um runbook seguro. Guardar limites de retenção e permitir reconciliação de jobs aceitos que ficaram sem evolução.

SEGURANÇA E PRIVACIDADE

Identidade e baseline de segurança

Preservar a trilha JWT da base, com ASP.NET Core Identity para usuários, hashing e recuperação. Usar bibliotecas de token validadas; não escrever criptografia própria. Atenção: os bearer tokens dos endpoints nativos do Identity não são JWTs padrão. A emissão JWT do laboratório deve ser explícita e separada. [B0, p. 5; R12]

Risco	Controle proposto
Roubo/reuso de credencial	Access token curto em memória; refresh em cookie HttpOnly/Secure, rotativo, hash no banco e família revogável. Nunca localStorage.
JWT inválido	Validar assinatura, algoritmo permitido, issuer, audience, expiração e chave. Não confiar apenas no payload decodificado.
CSRF / origens	Preferir mesma origem local para UI/API; refresh/logout com cookie exigem proteção CSRF. CORS restrito não substitui autorização.
Acesso por ID alheio	Ownership em consultas e comandos, inclusive versões, pesquisas, categorias e publicações.
XSS / conteúdo rico	Schema JSON allowlist, renderização segura e sanitização server-side quando houver HTML. Restringir protocolos de links.
Abuso e exposição	Rate limit de login/reset/pesquisa; corpo limitado; secrets fora do Git; diagnóstico por permissão de operador.

Recuperação desde cedo

Solicitar e-mail com resposta genérica para conta existente ou não. Gerar token de reset curto, específico, expirable e de uso único efetivo; enviar pelo SES local e nunca devolvê-lo na API pública. Validar expiração/reuso, alterar senha, invalidar o token e revogar sessões conforme a política. Não logar tokens nem revelar o cadastro. [R13]

Na trilha inicial, exercitar o fluxo com FakeEmailSender em testes e SES no LocalStack para integração. Inspecionar a mensagem capturada localmente; isso não demonstra entrega real em uma caixa postal. Preferir dados fictícios no laboratório. [R28]

Decisão comparativa para o ADR

Cookies de sessão com Identity são uma alternativa mais simples para uma SPA de navegador. O JWT aqui preserva o objetivo de estudo da base; documente a complexidade adicional, sem apresentar um emissor caseiro como um servidor OAuth/OIDC completo. [R12]

SEGURANÇA E PRIVACIDADE

IA, dados privados e conteúdo externo

Dúvidas podem revelar dados sensíveis. O provider recebe somente o trecho e o contexto necessário, após consentimento; nunca a conta inteira por conveniência. Uma nota privada no Notee não significa que seja seguro enviar seu texto a qualquer fornecedor.

Ameaça	Controle proposto
Prompt injection em página web	Tratar páginas como dados não confiáveis. Ferramentas limitadas; sem acesso a notas alheias, arquivos internos, credenciais ou publicação. [R15]
SSRF ao buscar URLs	Bloquear loopback, redes privadas, link-local e metadata endpoints; validar DNS/IP e cada redirecionamento. Restringir esquemas, tamanho e timeout. [R14]
Exfiltração pelo termo de busca	Minimizar a consulta, remover identificadores desnecessários, permitir bloqueio externo por nota e revisão do escopo.
Resposta que executa ações	Saída estruturada sem ferramentas de escrita. A resposta não pode criar usuário, publicar nota ou consultar secrets.
Vazamento por observabilidade	Logs/traces sem texto, fontes integrais, e-mails, prompts e tokens. Debug com conteúdo desabilitado por padrão.
Dados persistidos sem fim	Retenção curta de evidência bruta, respostas/revisões com política explícita, exclusão propagada e backups controlados.

Isolamento entre redes

O Worker precisa acessar LocalStack e PostgreSQL, mas a função que recupera páginas não deve poder usar essa conectividade para ler endpoints internos. Separar cliente/fetcher, aplicar validação de destino e, na etapa avançada, controles de egress. Um filtro textual na URL não resolve DNS rebinding. [R14]

Antes de habilitar um provider real, registrar quais dados saem, para quem, com qual retenção e quais limites. Não afirmar conformidade legal ou ausência de retenção sem analisar os termos aplicáveis. A arquitetura permite trocar o fornecedor, mas a troca precisa respeitar o consentimento anterior.

Conjunto de avaliação

Criar casos fictícios com fonte maliciosa, URL interna, afirmação sem suporte, fontes contraditórias e contexto insuficiente. Testar o comportamento de contenção; não apenas a qualidade aparente do texto gerado.

QUALIDADE

Testes de backend e verificação manual

Suíte	Ferramentas / dependências	Casos que importam
Domain	xUnit; sem infraestrutura.	Ciclo de vida, categoria/tag, revisão, âncora e matriz de automação.
Application	Fakes de relógio, providers e portas.	Consentimento, idempotência, limites, contexto, cancelamento e roteiro de pesquisa.
Integration	WebApplicationFactory + PostgreSQL em Testcontainers.	Auth, reset, migrations, ownership, concorrência real, constraints e contratos HTTP.
Messaging	LocalStack SQS + Worker; suíte dedicada.	Outbox, duplicatas, leases, checkpoints, retry, poison message e DLQ.
Architecture	Regras simples de dependência.	Domínio sem EF/AWS; acesso entre módulos somente por contratos.
Pesquisa / avaliação	Fixtures determinísticas; revisão humana separada.	SourceId inválido, Sem fonte, suporte parcial, contradição e injection.

Testcontainers permite exercitar integrações contra dependências em containers. Usar PostgreSQL real nos testes de SQL; um banco in-memory não comprova transações, índices e concorrência do ambiente-alvo. Isolar bancos/schemas entre testes paralelos. [R16]

Frontend: sem automação de testes nesta etapa

Não adicionar Vitest, React Testing Library ou Playwright ao pipeline do produto agora. Manter build, typecheck e lint, que não substituem testes de comportamento. Fazer roteiro manual: entrar/recuperar, editar, marcar palavra/frase, aceitar/ignorar sugestão, alternar automação, visualizar Sem fonte, publicar e retirar.

Verificar teclado e foco do modal, status anunciados, seleção preservada, erros legíveis, conflito sem perda do buffer, formulários sem envio duplo e leitura pública sem dados privados. Registrar data, navegador e resultado do teste manual. A validação visual do HTML entregue não cria uma obrigação de testes automatizados de frontend no roadmap.

Sinal de qualidade

Priorize cenários que protegem dados e garantias. Uma meta arbitrária de 100% de cobertura não substitui testar ownership, falhas entre commits e reexecuções concorrentes.

INFRAESTRUTURA E ENTREGA

Ambiente local e perfis de execução

Tudo que representa um serviço AWS no projeto será executado contra o LocalStack. A meta não é provisionar uma conta AWS real. Nem todos os componentes são serviços AWS: React, API, Worker, ferramentas de teste e observabilidade continuam sendo software local.

Perfil proposto	Composição	O que demonstra
learn-core	API e frontend locais; PostgreSQL em container; FakeResearchProvider.	Regras, SQL, auth e experiência sem depender de IA comercial ou da emulação completa.
async-local	API/Worker + PostgreSQL/Redis locais; SQS, DLQ e SES no LocalStack; telemetria local.	Entrega durável e integrações selecionadas, com infraestrutura explicitamente mista.
aws-full-local	Terraform provisiona SQS, SES, S3, Secrets Manager, RDS, ElastiCache, ECR/ECS e ALB no LocalStack conforme plano/cobertura.	Alvo de laboratório para APIs AWS e deploy local. Não comprova comportamento integral da AWS.
research-live (opt-in)	Um provider real de busca/IA externo, com consentimento e teto de uso.	Integração real. Não é emulada pelo LocalStack e pode gerar custo.

O perfil learn-core é um degrau de aprendizado, não um substituto silencioso do alvo LocalStack. Se o plano não cobrir RDS/ECS/ALB, documentar o bloqueio e manter o marco correspondente pendente, ou aprovar explicitamente uma alternativa. Não chamar PostgreSQL de Docker de “RDS validado”.

Convenções operacionais

Versões fixadas de imagens; .env.example sem secrets; volumes de dados separados dos dados descartáveis; binds apenas em loopback quando possível. Documentar nomes de host: localhost no host não significa localhost dentro do container. API/Worker usam o endpoint acessível pela rede Docker.

Reset não pode ser surpresa

Separar comandos de subir, parar, resetar somente infraestrutura temporária e apagar dados. Qualquer destroy com volume de banco ou state deve pedir confirmação e explicar o que será perdido.

INFRAESTRUTURA E ENTREGA

LocalStack: capacidade e limites reais

Checagem documental em 30/09/2026: as imagens atuais do LocalStack exigem auth token; desde 23/03/2026 há uma imagem unificada e o acesso a serviços depende da licença associada. Não assumir que todo o desenho AWS seja gratuito ou disponível no plano utilizado. Validar a cobertura de cada operação antes de fechar a implementação. [R19, R20]

Serviço	Uso no Notee	Limite de interpretação
SQS + DLQ	Transporte dos jobs e laboratório de falhas.	Testar operações usadas; emulação não substitui o contrato AWS.
SES	Recuperação de senha capturada localmente.	Mensagem capturada não comprova entrega externa. [R28]
RDS PostgreSQL	Banco do perfil full-local.	Disponibilidade por plano e limitações de persistência/gestão; não demonstra Multi-AZ real. [R22]
ElastiCache	Cache no perfil full-local.	Plano e cobertura; a documentação lista limites em failover, TLS e outras operações. [R23]
ECR + ECS	Registry e tarefas API/Worker.	ECS local executa containers no Docker; não prova isolamento ou capacidade real do Fargate. [R24, R26]
ALB	Roteamento do deploy local.	Validar APIs e integração de targets utilizadas; não afirmar paridade completa. [R25]
S3	Exportações/backups de laboratório, quando adicionados.	Objeto local não prova durabilidade da AWS. Não forçar upload como feature. [R27]
Secrets Manager	Exercitar obtenção de configuração.	Segredo de laboratório não tem garantia equivalente ao serviço gerenciado. [R29]

Planos, entitlements e cobertura podem mudar. Antes do marco full-local, produzir uma matriz com operação AWS, versão do LocalStack, plano, teste executado e limitação observada. A documentação de RDS e ElastiCache consultada os coloca nos planos Base/Ultimate; verificar a conta real em vez de inferir acesso. [R22, R23]

Honestidade no portfólio

Escrever “AWS emulada com LocalStack; cenários X/Y validados”. Não escrever “produção AWS validada”, “Multi-AZ testado” ou “sem custo” sem evidência.

INFRAESTRUTURA E ENTREGA

Terraform e deploy exclusivamente local

Terraform descreve os recursos; o LocalStack oferece endpoints de emulação. A documentação atual recomenda o comando lstk terraform ou configuração explícita de endpoints. O wrapper tflocal ainda pode existir, mas foi sucedido pelo novo fluxo. Fixar a versão escolhida e testar o procedimento no ambiente do aluno. [R21]

Etapa de aprendizado	Entrega esperada
1. Bootstrap seguro	Subir LocalStack e validar licença. Configurar somente credenciais fictícias AWS e endpoints locais. O auth token do LocalStack é real e fica fora do Git.
2. Infraestrutura mínima	Criar fila/DLQ, política de redrive e identidade de e-mail local. Produzir outputs consumidos pela aplicação.
3. Imagens	Build local com tag de commit; registrar/push no ECR emulado, conforme cobertura.
4. Full-local	Adicionar banco/cache emulados, task definitions, serviços e ALB. Executar smoke tests em ordem conhecida.
5. Evolução	Plan revisado, apply controlado, migration em etapa única, health checks e teste de rollback de aplicação.
6. Limpeza	Destroy somente do ambiente confirmado. Backup/restore de dados tem procedimento separado.

Proteções contra sair do laboratório

Não fornecer credenciais AWS reais aos scripts. Validar host/endpoint e conta local esperada antes de apply; uma checagem só do número da conta não substitui a validação de endpoints. Desabilitar busca de credenciais de metadata e evitar herança de perfis pessoais. Credenciais, tokens e state não entram no repositório.

Sensitive em Terraform oculta apresentação, mas valores ainda podem constar em state/plan. Preferir provisionar metadados de secrets e injetar valores de laboratório por etapa controlada. Guardar state em local protegido; não publicar arquivos de plano como artifacts sem revisão. [R30]

Não antecipar uma plataforma inteira

Começar com poucos arquivos legíveis. Extrair módulos Terraform quando houver repetição real. O objetivo é conseguir explicar um plan, um output, o state e uma falha de apply, não criar abstrações antes de usar o serviço.

INFRAESTRUTURA E ENTREGA

CI/CD enxuto e verificável

Iniciar a integração contínua cedo. A primeira versão do pipeline precisa dar feedback rápido e reproduzível, sem inserir uma ferramenta para cada prática avançada. As etapas abaixo são o contrato do pipeline a implementar, não arquivos já gerados neste pacote.

Gatilho	Etapas obrigatórias
Pull request	Restore com locks; build backend; testes unitários e integração PostgreSQL; npm ci; typecheck/lint/build do frontend.
Mudanca em infra	terraform fmt/check + validate sem credenciais reais. Teste de plan no LocalStack em job dedicado quando configurado.
Merge / release de laboratório	Build de imagens imutáveis; artifact com hashes/versões. Deploy local controlado e smoke tests.
Manual / integração dedicada	Outbox/SQS/Worker/SES no LocalStack e cenários de falha. Provider de IA Fake, sem consumo pago.

O que significa CD neste projeto

Alvo: provisionamento e atualização automatizáveis do ambiente local. Pode haver um job efêmero que cria LocalStack, aplica infra, sobe a aplicação, testa e destrói tudo. Isso valida a entrega local; não cria um staging persistente acessível depois que o runner termina.

No computador do aluno, um script ou comando de release aplica a versão aprovada. Não expor runner self-hosted pessoal a código de PR não confiável. Separar validação sem secrets de operações com token; se LocalStack exigir CI auth token, usá-lo como secret e respeitar o plano. [R19]

Evoluções posteriores, não bloqueios iniciais

Dependabot, análise de vulnerabilidades, CodeQL, SBOM, assinatura de imagens e ambientes efêmeros por PR podem entrar quando o pipeline básico estiver compreendido. Não adicionar testes frontend agora. Segredos fora do Git, permissões mínimas e dependências fixadas continuam sendo baseline, não opcionais.

Definition of Done de entrega

Um checkout limpo passa no build/teste, tem instruções executáveis, imagens identificadas por commit e um caminho conhecido de rollback. Migration incompatível não é resolvida simplesmente voltando a imagem anterior.

OPERAÇÃO

Observabilidade e diagnóstico

Instrumentar API e Worker com OpenTelemetry para logs, métricas e traces enviados a um backend local. O material original já previa OpenTelemetry e otel-lgtm; preservar essa direção e acrescentar sinais do fluxo de pesquisa. Não confundir uma tela de diagnóstico ilustrativa com instrumentação real. [B0, p. 6; R17]

Sinal	Pergunta operacional
HTTP duration / error rate	A API responde rapidamente para aceitar jobs ou está esperando o provider?
research_jobs_total{status,trigger}	Quantas pesquisas foram aceitas, concluídas, canceladas ou falharam?
research_job_duration_seconds	Qual o tempo de conclusão? Separar espera em fila de trabalho ativo.
research_sources_count / no_source_total	Há aumento de respostas sem evidência ou falhas de recuperação?
outbox_pending / oldest_age_seconds	Eventos ficaram presos antes de chegar ao SQS?
queue_age / retry_total / dlq_depth	O gargalo é consumo, indisponibilidade externa ou mensagem inválida?
provider_tokens / cost_estimated	Quanto foi reportado ou estimado? Distinguir estimativa, valor real e uso desconhecido.
lease_expiration / duplicate_events	A concorrência e a deduplicação estão funcionando como esperado?

Correlação via trace context no envelope; JobId e EventId em logs estruturados com acesso controlado, não como labels de métricas. Evitar cardinalidade alta por usuário, nota, URL e prompt. Registrar códigos de erro e provider/modelo controlados, sem guardar texto privado nos spans.

Metas de laboratório propostas

Hipóteses iniciais: aceitação de job p95 inferior a 500 ms sob carga declarada; zero job confirmado perdido nos cenários de falha testados; toda resposta classificada como Sourced com citações estruturalmente válidas. Tempo de resposta da pesquisa depende da web/provider e deve ser medido, não prometido.

Health sem efeito cascata

Liveness indica processo vivo; readiness indica dependências necessárias para atender aquela operação. Uma falha de IA não deve tirar do ar o CRUD de notas. Definir degradação parcial e sinalizar claramente o recurso indisponível.

OPERAÇÃO

Escala, busca, custo e recuperação

Melhorar o gargalo que foi medido

API sem estado de sessão em memória permite múltiplas instâncias; persistir chaves de proteção/configuração de forma consistente. Worker escala por concorrência limitada e lease. Antes de aumentar consumidores, verificar pool de conexões, limites do provider e orçamento; backlog não autoriza gasto ilimitado.

Tema	Decisão inicial e evolução
Cache	Adicionar Redis somente após medir. Cache-aside de recursos simples com owner/version na chave; não cachear todas as combinações de busca. Autorização não vem do cache.
Busca textual	Começar simples; evoluir para tsvector/GIN e configuração de idioma apropriada. Medir EXPLAIN ANALYZE e ranking no corpus de notas fictícias. [R18]
Busca semântica	Roadmap opcional: embeddings/pgvector por owner e revisão, custo de reindexação, exclusão e comparação de qualidade com FTS. Não necessária para pesquisar na web.
Quota e custos	Reservar capacidade atomicamente por conta. Concorrência, consultas, tokens e timeout limitados. Valores estimados ficam identificados como estimativas.
Dados e retenção	Proposta: lixeira por 30 dias com prazo visível; evidência bruta por 7 dias quando permitido; revisões/respostas até exclusão da nota ou limite comunicado.

Runbooks que precisam ser exercitados

Fila parada: identificar etapa, restaurar dependência, verificar Outbox e backlog. Provider indisponível: abrir circuito, limitar retries, avisar usuários e manter notas funcionando. Exclusão: revogar publicação, cancelar trabalhos e limpar dados associados com tarefa idempotente; evitar ressuscitar conteúdo por resposta tardia.

Backup: exportar PostgreSQL e guardar cópia de laboratório fora do volume de origem; testar restore em banco separado. Snapshot do LocalStack não substitui esse teste. Registrar RPO/RTO medidos e a política de expiração dos backups; não prometer eliminação imediata de toda cópia histórica.

Teste de escala honesto

Usar dados sintéticos e carga documentada, por exemplo 10 usuários concorrentes e 100 mil notas. Reportar máquina, configuração e percentis. Resultado local não é capacidade garantida de produção.

TRILHA DE IMPLEMENTAÇÃO

Marcos 01 a 03

Sem datas e sem estimativa de horas. Avançar pelo critério de aceite; os marcos são unidades de aprendizado, não sprints de duração fixa. Cada sessão deve trabalhar apenas uma pequena parte de um marco.

M01 | Base reproduzível

Depende de: Nenhuma. Preparar repositório, .NET, Node, VS Code e convenções. Confirmar acesso, auth token e cobertura do plano LocalStack antes de assumir o laboratório completo. Registrar versões sem exigir as mais recentes a cada aula.

Mão na massa: Criar a solution e o primeiro teste de domínio; executar build em checkout limpo. Escrever o README inicial e uma CI mínima de build/teste backend.

Aceite: O autor explica o papel de cada projeto, executa o teste sem IA e sabe dizer quais serviços locais ainda não foram validados.

Orientação da mentoria

Começar pequeno; não gerar todas as pastas e interfaces de uma vez.

M02 | Domínio de notas

Depende de: M01. Modelar Note, Category, Tag e os estados Active, Archived e Trashed. Definir invariantes de ownership e o que muda a versão.

Mão na massa: Implementar criação, edição, arquivamento e lixeira com testes. Rejeitar categorias/tags de outro owner na fronteira apropriada.

Aceite: Os testes mostram transições válidas e inválidas, sem depender de ASP.NET ou banco. RF02 e RF03.

Orientação da mentoria

Explicar entidade, value object e serviço de aplicação usando o código que o aluno escreveu.

M03 | API e persistência

Depende de: M02. Introduzir ASP.NET Core, casos de uso, EF Core, PostgreSQL, migrations e Problem Details.

Mão na massa: Persistir notas e taxonomia; testar endpoints com PostgreSQL real em container. Implementar o compare-and-swap da versão e a primeira chave de idempotência.

Aceite: Duas atualizações simultâneas não se sobrescrevem; erro HTTP é previsível. A API local não é exposta publicamente antes da autenticação. RF11.

Orientação da mentoria

Explicar transação e constraint antes de sugerir abstração de repositório.

TRILHA DE IMPLEMENTAÇÃO

Marcos 04 a 06

M04 | Identidade e recuperação

Depende de: M03. Adicionar e-mail/senha e recuperação desde cedo. Separar identidade autenticada de dados fornecidos pelo cliente.

Mão na massa: Usar Identity para credenciais/reset e implementar a decisão de sessão documentada. Exercitar reset com FakeEmail e depois caixa local SES; aplicar rate limit e ownership.

Aceite: Usuário A nunca lê/edita dado de B. Resposta de recuperação não enumera contas; token expirado/reutilizado falha; reset encerra sessões conforme política. RF01.

Orientação da mentoria

Não ensinar hashing próprio nem guardar senha/token no frontend.

M05 | Design system e primeira tela

Depende de: M04. Criar React/Vite/TypeScript e transformar tokens e componentes do HTML em componentes Notee.

Mão na massa: Implementar shell, botões, inputs, badges e listagem de notas. Conectar autenticação e API; representar loading, vazio, erro e acesso negado.

Aceite: Uma nota criada pela API aparece na interface; foco e navegação por teclado são inspecionados manualmente. Sem testes automatizados frontend nesta fase.

Orientação da mentoria

Distinguir estado de servidor, estado do formulário e estado visual; não copiar app.js inteiro para React.

M06 | Editor, revisões e seleção

Depende de: M05. Adotar editor de blocos com IDs estáveis e revisões completas. Tornar o salvamento previsível antes de automatizá-lo.

Mão na massa: Começar com salvar explícito; serializar autosave depois. Capturar palavra/frase com posição UTF-16, quote e contexto. Implementar restauração como nova revisão.

Aceite: Um conflito nunca perde texto local; restaurar não apaga histórico; palavra com emoji/acentos preserva a seleção. RF04 e RF11.

Orientação da mentoria

Explicar por que offsets sozinhos quebram quando o documento muda.

TRILHA DE IMPLEMENTAÇÃO

Marcos 07 a 09

M07 | Busca e organização

Depende de: M06. Fechar pesquisa interna, categorias, tags, arquivo e lixeira. Separar essa busca da pesquisa na internet.

Mão na massa: Implementar filtros, paginação e índices. Evoluir para FTS quando houver corpus para comparar consultas e relevância.

Aceite: Uma consulta nunca retorna notas privadas de outro owner; filtros combinados funcionam; plano de execução é registrado com dados sintéticos. RF02, RF03 e RF12.

Orientação da mentoria

Não introduzir embeddings nem cache antes de entender a consulta SQL.

M08 | Publicação revisada

Depende de: M07. Implementar snapshot público/não listado independente do rascunho. Manter o produto sem rede social.

Mão na massa: Criar preview, publicação, atualização explícita e revogação. Renderizar página pública, perfil e biblioteca de leitura.

Aceite: Editar a nota não altera snapshot; não listada não aparece no catálogo; despublicar/lixeira impede novas leituras. RF10.

Orientação da mentoria

Explicar a diferença entre privado, URL não listada e autorização real.

M09 | Dúvidas e contrato da pesquisa

Depende de: M06. Modelar dúvida, tentativas, fontes, revisão humana e estados sem depender de um fornecedor de IA.

Mão na massa: Implementar marcação manual, refinamento e contrato 202 + JobId. Usar FakeResearchProvider e fixtures; manter a aplicação utilizável sem web.

Aceite: O fluxo mostra pendente, concluída, erro, cancelada e Sem fonte; resposta não substitui texto original. RF04, RF07, RF08 e RF09.

Orientação da mentoria

Não usar Task.Run como substituto da etapa durável que vem em seguida.

TRILHA DE IMPLEMENTAÇÃO

Marcos 10 a 12

M10 | Sugestões e automação

Depende de: M09. Introduzir análise de texto e sugestões de dúvida, tag e categoria com aceite.

Mão na massa: Implementar precedência conta/nota, bloqueio global e debounce. Começar com análise Fake versionada; cancelar sugestões obsoletas e evitar duplicatas.

Aceite: Nota Off não executa automaticamente; Inherit segue a conta; sugestão não aceita não vira pesquisa. Mudança de preferência não dispara backlog antigo. RF05 e RF06.

Orientação da mentoria

Explicar diferença entre detectar intenção, confirmar pergunta e autorizar gasto externo.

M11 | Outbox e SQS no LocalStack

Depende de: M09; acesso validado em M01. Tornar persistência do comando e intenção de processamento atômicas.

Mão na massa: Gravar job e Outbox na mesma transação; publicar no SQS local por dispatcher. Consumir mensagens duplicadas de forma idempotente e encaminhar falhas permanentes para DLQ.

Aceite: Parar SQS não perde o job confirmado; restaurá-lo esvazia Outbox; duplicata não cria resultado extra. RF07.

Orientação da mentoria

Mostrar primeiro a falha do dual write; depois explicar por que Outbox resolve apenas parte do problema.

M12 | Worker recuperável

Depende de: M10 e M11. Implementar leases, checkpoints, retries, cancelamento e limites de concorrência.

Mão na massa: Executar FakeResearch em etapas duráveis. Injetar falhas após busca, antes do commit e antes do ACK. Usar fencing token para impedir gravar resultado de lease vencido.

Aceite: Reinício retoma etapa persistida; resultado tardio cancelado não é anexado; timeout externo ambíguo fica explícito. RF07 e RF13.

Orientação da mentoria

Não prometer exactly-once em chamada externa nem manter transação aberta durante a pesquisa.

TRILHA DE IMPLEMENTAÇÃO

Marcos 13 a 15

M13 | Pesquisa real com fontes

Depende de: M12. Implementar um provider real por vez atrás dos contratos, com opt-in de dados/custo.

Mão na massa: Buscar na web, validar URLs/citações e sintetizar resposta vinculada a evidências. Registrar modelo, promptVersion e uso reportado; limitar contextos e orçamento.

Aceite: Sem fonte aparece quando não há evidência; fonte inacessível não é tratada como lida. A troca por Fake continua possível. RF08 e RF09.

Orientação da mentoria

Comparar capacidades dos providers em vez de prometer intercambialidade total.

M14 | Segurança e avaliação

Depende de: M13. Aprofundar a segurança iniciada na autenticação, sem tratar hardening como correção final de um desenho inseguro.

Mão na massa: Testar SSRF, XSS, prompt injection, acesso cruzado, quotas e revogação. Montar corpus de dúvidas ambíguas, fontes conflitantes e fontes ausentes.

Aceite: URLs locais/redirecionadas são bloqueadas; nota privada não vaza no prompt/log/público; limitações da resposta são visíveis.

Orientação da mentoria

Separar teste determinístico do pipeline de avaliação de qualidade do modelo.

M15 | Observabilidade e falhas

Depende de: M12; M13 opcional live. Demonstrar o caminho da requisição até o resultado usando traces e métricas locais.

Mão na massa: Instrumentar API/Worker/HttpClient. Construir painéis de fila, erros, Sem fonte e custo estimado. Executar os cenários do laboratório de falhas.

Aceite: Um JobId é rastreável sem expor texto privado; idade da fila e Outbox revelam um gargalo; CRUD segue funcionando com IA indisponível.

Orientação da mentoria

Pedir uma hipótese de diagnóstico apenas se o aluno solicitar exercícios em formato de perguntas.

TRILHA DE IMPLEMENTAÇÃO

Marcos 16 a 18

M16 | Terraform e AWS full-local

Depende de: M11 e cobertura/plano confirmados. Provisionar somente no LocalStack os serviços AWS definidos para o laboratório.

Mão na massa: Adicionar SQS/DLQ, SES, Secrets Manager, ECR, ECS, ALB, RDS e ElastiCache conforme cobertura. Conectar aplicação aos endpoints/outputs locais; S3 somente se houver uso real.

Aceite: Plan/apply/destroy reprodutíveis, smoke tests por serviço e nenhuma credencial/conta AWS real. Se a licença bloquear, o marco fica explicitamente pendente, sem fingir paridade.

Orientação da mentoria

Explicar cada recurso antes de extrair um módulo Terraform. Esta etapa não pode invalidar o aprendizado já feito.

M17 | Entrega, desempenho e restore

Depende de: M15 e M16. Fechar CI/CD leve, experiências de escala e recuperação com evidências reproduzíveis.

Mão na massa: Testar release local e rollback de imagem compatível; restaurar banco separado. Medir gargalo e só então adicionar Redis em uma leitura adequada.

Aceite: Um checkout limpo constrói e testa; relatório declara carga/máquina; cache não vaza acesso e invalida corretamente; restore medido.

Orientação da mentoria

Manter frontend em typecheck/lint/build e checklist manual, sem adicionar suíte automatizada agora.

M18 | Evidências e consolidação

Depende de: Marcos anteriores concluídos ou pendências declaradas. Transformar o projeto em um portfólio honesto que o autor consegue explicar e depurar.

Mão na massa: Revisar ADRs, README, diagramas, limites e demo. Registrar problemas encontrados e decisões revistas. Atualizar o estado do projeto para outra IA continuar sem inventar progresso.

Aceite: Demonstração une nota, dúvida, fila, resposta, fonte e publicação; uma falha e sua recuperação são reproduzidas ao vivo.

Orientação da mentoria

Evoluções como responsividade, testes frontend, busca semântica ou rede social exigem nova decisão de escopo.

APRENDIZADO E EVIDÊNCIAS

ADRs: decisões que você deve defender

ADR é um registro curto de decisão arquitetural: contexto, opções, escolha, consequências e sinal para reconsiderar. Não precisa de uma tese nem de formalidade para toda linha de código. As decisões abaixo merecem documentação porque têm alternativas e custos reais.

ADR	Decisão inicial / quando reconsiderar
001 | Monólito	Módulos por negócio e API/Worker separados. Reconsiderar serviços quando houver necessidade demonstrada de isolamento ou equipes independentes.
002 | SPA + API	React/Vite com .NET separado. Reavaliar SSR/prerender se a descoberta por buscadores das publicações virar requisito central.
003 | Identidade	Identity + sessão explicitamente desenhada. Comparar cookies e JWT; não chamar token proprietário de JWT.
004 | Conteúdo	Blocos com IDs e schema versionado. Avaliar migração do schema antes de adicionar tipos de bloco.
005 | Publicação	Snapshot separado e revogável. Reavaliar somente se autor aceitar atualização pública automática.
006 | Pesquisa	Providers com capacidades declaradas; resultado separado com fontes. Integração nativa web é adaptador, não contrato do domínio.
007 | Entrega durável	Outbox + SQS + checkpoints PostgreSQL. Registrar retries, leases e limite da garantia externa.
008 | Cache / FTS	PostgreSQL como fonte de verdade. Redis entra depois de medição; embeddings são opção futura.
009 | LocalStack	AWS somente emulada. Plano/cobertura são pré-condições do perfil full-local, sem declarar paridade de produção.
010 | Qualidade	Backend automatizado; frontend manual e build tipado agora. Documentar lacuna, sem afirmar cobertura E2E que não existe.

Sinal de profundidade

Conseguir contar qual problema apareceu, por que a alternativa simples deixou de atender, como a escolha foi testada e que custo ela trouxe. Um projeto pessoal pode demonstrar isso sem se apresentar como experiência de produção que não ocorreu.

INSTRUÇÕES PARA CONTINUAR COM IA

Contrato da IA mentora

Estas instruções se aplicam ao desenvolvimento futuro do Notee. A IA deve atuar como mentora, não como geradora autônoma de uma aplicação inteira. O objetivo é ampliar a capacidade de implementar, raciocinar, testar e depurar do autor.

Comportamento	Regra
Antes da implementação	Ler o planejamento e o estado atual; explicar o objetivo e o motivo técnico da próxima tarefa. Não inventar arquivos existentes.
Escopo por resposta	Trabalhar uma tarefa pequena com resultado observável. Não entregar todos os marcos, backend ou frontend de uma vez.
Dicas	Começar com conceitos, contratos e exemplos menores/isolados. Dar pistas graduais e esperar uma tentativa.
Código	Não escrever implementação completa por padrão. Só fornecê-la quando o autor pedir explicitamente; delimitar arquivo e objetivo.
Revisão	Ler o código enviado, apontar riscos e um caminho de correção. Não substituir tudo sem explicar o defeito.
Motivos e perguntas	Explicar os motivos diretamente. Só fazer perguntas pedagógicas antes da explicação quando o autor solicitar. Perguntas necessárias para resolver ambiguidade continuam permitidas.
Testes e depuração	Orientar cenários e critérios de aceite; pedir saídas reais. Nunca afirmar que executou algo que não executou.
Fontes e versões	Conferir documentação oficial quando o comportamento depender de versão. Separar decisão proposta de fato documentado.
Continuidade	Encerrar a sessão atualizando progresso, evidências, decisões e próxima tarefa. Não avançar por mera aparência de conclusão.

Formato padrão de uma aula

Objetivo da tarefa → por que ela existe → conceitos essenciais → passo pequeno para o autor implementar → critério para verificar → pausa para receber a tentativa. Comparar com JavaScript/TypeScript quando isso facilitar entender C#/.NET.

INSTRUÇÕES PARA CONTINUAR COM IA

Prompt mestre para iniciar uma sessão

Copie o texto abaixo e anexe o planejamento e o estado do projeto. Há uma cópia pronta em docs/MENTOR_IA.md. Se a IA tiver acesso ao repositório, ela deve inspecioná-lo antes de inferir o que foi implementado.

Você será minha mentora de desenvolvimento no projeto Notee.

Leia primeiro o planejamento e o arquivo ESTADO_DO_PROJETO.md. O PDF/Markdown define o produto-alvo; o HTML é apenas um protótipo visual, com dados e pesquisas simulados. Não pressuponha que existe backend, React ou infraestrutura implementados.

Quero colocar a mão na massa. Oriente uma tarefa pequena de cada vez: explique o objetivo, os motivos e os conceitos, proponha o que devo implementar e como verificar, e aguarde meu código ou resultado. Dê dicas graduais e revise minha tentativa antes de oferecer a solução. Não gere toda a implementação, scaffolding extenso ou arquivos completos sem meu pedido explícito.

Explique os motivos diretamente. Não me submeta a perguntas de verificação antes da explicação, a menos que eu peça esse formato. Quando necessário, faça somente perguntas para resolver uma ambiguidade que impede o trabalho.

Preserve o escopo: React/TypeScript/Vite com API ASP.NET Core em monólito modular, PostgreSQL, Worker e LocalStack para toda AWS. Sem deploy AWS real, rede social, cronograma fixo, testes automatizados de frontend ou responsividade nesta etapa. Recuperação de senha entra cedo. Pesquisa web deve ter fontes reais ou indicar Sem fonte; nunca invente citações.

As sugestões de dúvidas, tags e categoria exigem aceite. A pesquisa automática respeita conta/nota e privacidade. Respostas e publicações não podem alterar silenciosamente o texto do autor. Explique idempotência, Outbox, retries, leases, segurança e trade-offs quando chegarmos a essas etapas, não como uma lista de tecnologias para copiar.

Use documentação oficial; identifique suposições e limitações do LocalStack. Não solicite senhas, tokens, dados de trabalho ou notas reais para exemplos. Nunca afirme que testes ou deploys passaram sem evidência. Ao final, atualize o estado do projeto com o que foi realmente feito.

Comece pela próxima tarefa ainda não concluída no estado do projeto. Se este for o início, use M01: explique a base e me guie no primeiro passo, sem gerar o sistema inteiro.

INSTRUÇÕES PARA CONTINUAR COM IA

Continuidade entre sessões e modelos

A memória da conversa não substitui o repositório. Use um arquivo pequeno e versionado para que qualquer IA entenda o estado real, sem receber todo o histórico e sem inventar avanços. Na primeira sessão, o status é: planejamento e protótipo entregues; aplicação-alvo ainda por implementar.

```
# Estado do Notee
Marco atual: M01
Tarefa atual: ...
Commit/base inspecionada: ...
Implementado e verificado: ...
Comandos executados e resultado: ...
Nao executado / ainda simulado: ...
Decisoes novas (ADRs): ...
Duvidas tecnicas abertas: ...
Falha atual + reproducao minima: ...
Proxima tarefa pequena: ...
Arquivos relevantes: ...
Modo de mentoria: explicar -> orientar -> aguardar
```

Como pedir revisão de código

Enviar o objetivo, o trecho relevante, o erro exato e os testes executados. Pedir que a IA aponte primeiro o problema e a menor correção possível. Para um arquivo grande, tratar uma responsabilidade por vez. Evitar compartilhar secrets, logs com conteúdo privado e arquivos de trabalho reais.

Como registrar a conclusão de um marco

Relacionar critério de aceite a uma evidência: teste automatizado backend, captura da interface, comando com saída ou experimento documentado. Anotar limitações e testes não feitos. Uma resposta convincente da IA não conta como execução do software.

Histórico honesto

Fazer commits pequenos durante a implementação real. Não fabricar uma narrativa de evolução retroativa, nem apresentar o protótipo gerado como frontend de produção escrito e validado pelo autor. O README deve dizer o que foi aprendido e implementado de fato.

ENTREGA E LIMITES

Como usar o protótipo entregue

Abra Notee_Prototipo.html no navegador desktop, ou extraia o pacote e abra prototipo/index.html. A versão de arquivo único incorpora CSS e JavaScript; a pasta separada facilita estudar e migrar os componentes. Nenhuma biblioteca remota, fonte externa, conta ou chave de API é necessária.

Roteiro manual	O que experimentar
1. Entrar	Abrir a landing e entrar na demonstração. Login/cadastro/recuperação são encenações, sem autenticação nem e-mail real.
2. Escrever	Criar nota, editar título/blocos, salvar e selecionar uma palavra ou frase. Clicar em Marcar como dúvida e confirmar a pergunta.
3. Analisar	Acionar análise; aceitar/rejeitar sugestões de dúvidas, tags e categoria. A heurística local não é um modelo de IA.
4. Pesquisar	Iniciar uma pesquisa simulada, ver progresso, abrir fontes ou Sem fonte, revisar resultado e testar cancelamento/falha.
5. Configurar	Alterar padrão da conta e override da nota. O bloqueio global tem precedência.
6. Publicar	Criar snapshot com ou sem respostas revisadas; editar rascunho e observar independência. Despublicar e visitar biblioteca/perfil.
7. Recuperar	Abrir revisões, arquivo e lixeira; testar restauração. Para recomeçar: Configurações > Privacidade e dados > Reiniciar dados.

Diferenças intencionais em relação ao produto-alvo

O editor usa contenteditable simplificado, não Tiptap. Pesquisas usam temporizadores locais, sem durabilidade ao fechar a página. Dados ficam no navegador e podem desaparecer; não há criptografia, backend, permissões reais, banco, Terraform ou LocalStack executando neste pacote. A tela de diagnóstico usa números demonstrativos.

O histórico e as tentativas de pesquisa do mock são simplificados: refinamentos podem substituir o resultado visual, e nem toda alteração auxiliar cria revisão. A implementação real deve seguir os registros imutáveis e regras deste plano. Links compartilhados permanecem locais; o HTML não hospeda publicações na internet.

Privacidade da demonstração

Usar somente dados fictícios. O protótipo não faz pesquisa externa; abrir um link de fonte é uma navegação voluntária do usuário. Se o navegador bloquear armazenamento em arquivo local, o estado pode existir apenas em memória.

APRENDIZADO E EVIDÊNCIAS

Aceite final e demonstração técnica

O projeto termina esta etapa quando o fluxo principal funciona, as fronteiras são seguras e as garantias declaradas foram demonstradas. Não é necessário adicionar rede social, microsserviços ou novos serviços para encerrar uma versão valiosa.

Dimensão	Evidência exigida
Produto	Conta/reset, notas, categorias/tags, palavra/frase como dúvida, sugestões, preferências, resposta separada, fontes/Sem fonte e publicação revisada.
Consistência	Conflito de versão reproduzido; restauração como nova revisão; POST repetido sem duplicação; snapshot independente do rascunho.
Assíncrono	Job confirmado sobrevive a falha; duplicata e retomada controladas; cancelamento e DLQ testados; limites externos declarados.
Segurança	Autorização por owner; reset sem enumeração; XSS/SSRF/injection mitigados; secrets e conteúdo privado fora de logs/repositório.
Operação	Trace do fluxo, métricas relevantes, runbook de falha, restore real de laboratório e resultados de carga documentados.
Entrega	CI leve passa; versionamento de imagens; release/rollback local; Terraform validado no perfil de LocalStack efetivamente disponível.
Aprendizado	ADRs, instruções de execução, limites e uma explicação sem depender da IA para cada decisão central.

Demonstração sugerida, sem roteiro decorado

Criar uma nota e selecionar uma palavra. Confirmar a pergunta, pesquisar e continuar escrevendo. Abrir resposta e uma fonte, mostrar também Sem fonte. Publicar somente o snapshot escolhido. Depois interromper um Worker, retomar a execução e mostrar no trace por que a nota não foi perdida. Encerrar com um ADR e uma limitação real.

Profundidade não é um título automático

Este projeto pode evidenciar evolução técnica. Ele não substitui experiência de operação em produção ou trabalho em equipe. A melhor apresentação conecta problema, decisão, teste, resultado e aprendizado, com honestidade sobre o que continua pendente.

FONTES E VALIDAÇÃO

Referências 01 a 10

B0 | 07_Projeto_NoteNest_DotNet_para_Portfolio(2).pdf, 8 páginas, fornecido pelo autor. Base de arquitetura e aprendizado; ampliada e modificada conforme as respostas aprovadas nesta conversa. Não foi usado como evidência de um sistema já implementado.

Referências oficiais consultadas em 30/09/2026. Os títulos abaixo são links para as fontes. Versões, planos, recursos e limites devem ser reconferidos na implementação. Requisitos de produto e metas de laboratório são decisões do Notee, não promessas dessas fontes.

R01 AbacatePay | referência pública de identidade

www.abacatepay.com

R02 .NET | releases e suporte

learn.microsoft.com

R03 React | criar uma aplicação

react.dev

R04 Vite | guia de configuração

vite.dev

R05 TanStack Query | estado de servidor

tanstack.com

R06 Tiptap | extensões próprias

tiptap.dev

R07 Tiptap | identificadores únicos

tiptap.dev

R08 OpenAI | web search e citações

developers.openai.com

R09 EF Core | concorrência otimista

learn.microsoft.com

R10 AWS | padrão transactional outbox

docs.aws.amazon.com

FONTES E VALIDAÇÃO

Referências 11 a 20

Referências oficiais consultadas em 30/09/2026. Os títulos abaixo são links para as fontes. Versões, planos, recursos e limites devem ser reconferidos na implementação. Requisitos de produto e metas de laboratório são decisões do Notee, não promessas dessas fontes.

R11 AWS SQS | entrega ao menos uma vez

docs.aws.amazon.com

R12 ASP.NET Core Identity | APIs e autenticação

learn.microsoft.com

R13 OWASP | recuperação de senha

cheatsheetseries.owasp.org

R14 OWASP | prevenção de SSRF

cheatsheetseries.owasp.org

R15 OWASP | prevenção de prompt injection

cheatsheetseries.owasp.org

R16 Testcontainers | .NET

dotnet.testcontainers.org

R17 OpenTelemetry | .NET

opentelemetry.io

R18 PostgreSQL | full text search

www.postgresql.org

R19 LocalStack | auth token

docs.localstack.cloud

R20 LocalStack | release 2026.03.0 e imagem unificada

blog.localstack.cloud

FONTES E VALIDAÇÃO

Referências 21 a 30

Referências oficiais consultadas em 30/09/2026. Os títulos abaixo são links para as fontes. Versões, planos, recursos e limites devem ser reconferidos na implementação. Requisitos de produto e metas de laboratório são decisões do Notee, não promessas dessas fontes.

R21 LocalStack | Terraform e endpoints

docs.localstack.cloud

R22 LocalStack | RDS e limitações

docs.localstack.cloud

R23 LocalStack | ElastiCache e limitações

docs.localstack.cloud

R24 LocalStack | ECS executado localmente

docs.localstack.cloud

R25 LocalStack | load balancing

docs.localstack.cloud

R26 LocalStack | ECR

docs.localstack.cloud

R27 LocalStack | S3

docs.localstack.cloud

R28 LocalStack | SES e captura de e-mails

docs.localstack.cloud

R29 LocalStack | Secrets Manager

docs.localstack.cloud

R30 Terraform | dados sensíveis e state

developer.hashicorp.com

Leitura do material

Referência visual não significa clonagem do site; documentação do LocalStack não significa teste em uma conta do aluno. O pacote entrega um plano e uma simulação de interface. Cada marco exige sua própria evidência de implementação.